Smart classroom occupancy and lighting

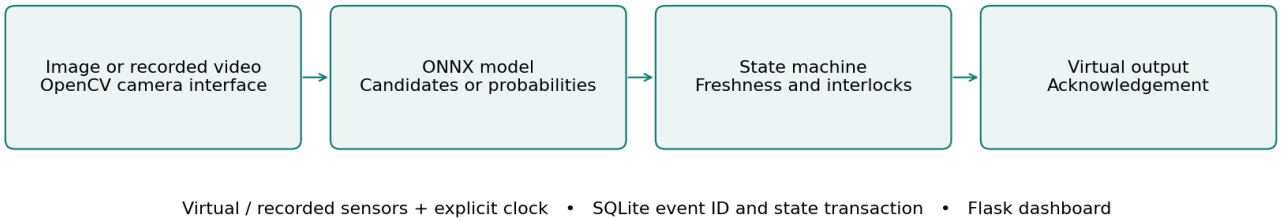
Technical report and reproducible evaluation

11 November 2026 | Abhijith Viswanathan | AI-assisted implementation

A camera candidate count drives a conservative lighting state machine. Motion and door events request new checks; absence of motion never establishes vacancy. The default recheck interval is 600 seconds, with a 120-second grace period and two negative observations before switch-off. Tracking provides line-crossing counts only when camera geometry supports that interpretation.

Raspberry Pi targeted, desktop simulation tested. No Pi, real sensor, electrical power, camera timing, motor or physical safety validation was performed. This is an AI-assisted personal engineering project; it does not establish professional seniority or independent authorship.

Architecture



The application keeps perception, control and hardware interfaces separate. Virtual inputs and the explicit clock make decisions replayable; SQLite commits event identifiers and controller state together. The dashboard runs locally and never opens a webcam unless explicitly enabled.

Main measured comparison

Person detection F1: 0.632 to 0.736 score. 78 COCO test images; IoU >= 0.5; threshold 0.25; 320 to 640 pixels.

The following pages separate real image evaluation, host execution time, scripted control outcomes and future physical validation. Raw observations, exact selections, source attribution and regression logs accompany this report.

Control behavior and recovery



State inventory; transition guards and recovery paths are defined in the policy table and core.py

1. Start in UNCERTAIN with lights on. Positive visual candidates retain occupied status even without motion.

2. An unreliable, disconnected or expired frame cannot confirm vacancy. The same stored image expires after two simulation seconds when a new check is requested.

3. Two negative checks and elapsed grace permit VACANT and lights off. Default polling can miss a short visit between checks; door and motion events reduce that delay when reliable.

4. Manual ON or OFF expires after 300 simulated seconds. Explicit manual OFF can darken an occupied room; it is logged and must remain a deliberate action.

Freshness and persistence

For classroom and parking, real image evidence expires after two simulation seconds at a new check; advancing time cannot make a stored frame fresh. Restart invalidates camera evidence. A gate interrupted during a cycle enters FAULT, and an interrupted waste move requires review. Manual simulation inputs are explicitly labeled fixtures.

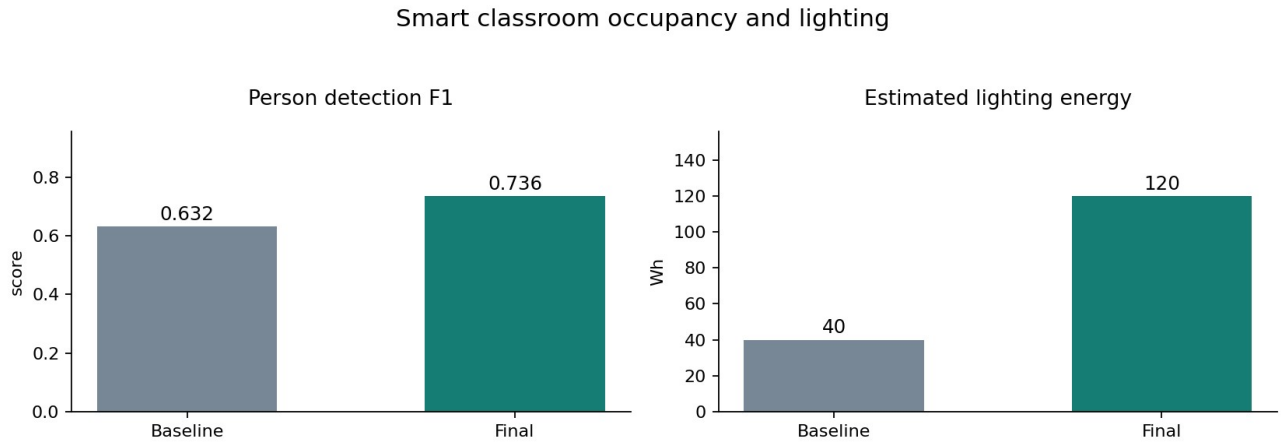
The classroom and parking tracker uses class-aware one-to-one overlap association, a five-second expiry, two-hit confirmation and a line deadband. An 80-frame synthetic trajectory produced one unique ID and one expected entry. This verifies mechanics, not tracking accuracy on crowded video. IDs and entry counters persist, but active tracks do not survive restart.

A state change and its event ID are stored in one SQLite transaction. Duplicate event IDs are ignored. Failed persistence rolls back in-memory controller state and newly accepted inference/tracker state. This cannot undo a real actuator movement; the physical design requires independent acknowledgement and reconciliation.

State-machine source: docs/state-machine.mmd. Controllers: lab/core.py. Hardware interfaces: lab/hal.py. The current app uses serialized, synchronous inference and virtual-clock control; it is not a real-time motor controller.

Models data and measured tradeoffs

YOLO11n weights are used as pretrained; no detector weights were trained or fine-tuned. ONNX exports use static 320 and 640 input sizes, float32, opset 17, CPU execution, two inference threads, argmax class selection and per-class NMS at IoU 0.45. Thresholds 0.15, 0.25, 0.40 and 0.55 were compared on 42 validation images; 0.25 was selected before evaluating 78 test images. Resolution is the measured model improvement; temporal and sensor policies are evaluated separately.



Higher resolution increased F1 from 0.632 to 0.736 but lowered precision from 0.867 to 0.829 and increased detector median host latency from 13.31 to 48.50 ms. The combined policy kept lights on throughout the difficult one-hour scripted trace: occupied-off time fell from 1200 to 0 seconds, while assumed energy rose from 40 to 120 Wh. This experiment does not demonstrate energy savings.

Nine of ten synthetic framed-photo composites still produced person candidates. The target for lighting is a physically present person, but COCO labels and the pretrained detector represent visual people; they do not establish liveness. Configured depiction zones mark ambiguity, and cannot discover every poster or screen. Tracking can also track a poster. No real classroom poster, seated-person, bags, crowd or screen-specific accuracy was measured. Image splits lack recording IDs; crowd annotations are excluded under a documented nonstandard metric.

Measurement protocol and verification

Metric	Baseline	Final	Unit
Person detection F1	0.632	0.736	score
Count mean absolute error	1.128	0.603	people/image
Occupied with lights off	1200.000	0.000	simulated seconds
Estimated lighting energy	40.000	120.000	Wh

Person detection F1: 78 COCO test images; IoU >= 0.5; threshold 0.25; 320 to 640 pixels.

Count mean absolute error: Same 78 images; visual labels, not verified physical presence.

Occupied with lights off: 3600-second scripted trace; default 600-second interval.

Estimated lighting energy: Assumed 120 W load; synthetic trace; no measured power.

Complete local inference requests: median 52.23 ms and p95 52.79 ms over 30 repeated requests after one cold request (75.75 ms). Flask test-client timing includes image decode, inference, annotation, controller update, SQLite and JSON; it excludes network and browser rendering. The same selected image was repeated. CPU and cumulative process memory samples are in evidence/request-timing.json.

Environment: macOS 26.2 arm64, Python 3.12.14, ONNX Runtime 1.21.0 CPU, OpenCV 4.11.0.86 and NumPy 2.2.6. Detector timing separately compares 36 calls at each size; median 13.31 ms at 320 and 48.50 ms at 640. Short host runs do not predict Pi throughput, thermal behavior or sustained response bounds.

Regression result: 56 shared cases passed, covering state guards, invalid and stale input, manual requests, duplicate IDs, restart, large time steps, failed commits and HTTP input boundaries. The same suite is included in each standalone project; these are not 168 distinct tests. Browser checks exercised real inference and the advertised failure controls.

Version history preserves the initial 38 passing core cases, subsequent API and review regressions, and the final 56-case run. Decoder correction, stale image handling, obstruction-source separation, capacity changing during motion, scenario labeling and restart/rollback fixes are recorded in docs/FAILURES.md. Earlier detection results remain available and are not used as final metrics.

Hardware integration and physical validation

Raspberry Pi 4 or 5 with 64-bit OS, USB camera, PIR motion sensor with verified 3.3 V output, door reed switch, LED and 330 ohm resistor, microSD, suitable Pi supply.

Example BCM allocation: LED GPIO17 through 330 ohm resistor to ground; motion GPIO27; door GPIO22 with pull-up. USB camera uses USB. Verify the PIR output voltage before connection. These are suggested pins, not a tested harness.

Interface	Implemented now	Required before physical use
Camera	Recorded files and explicit USB camera option	Capture cadence and disconnect validation
Sensor readings	Virtual and recorded value valid at records	Calibrated GPIO or serial acquisition
Outputs	VirtualOutput and optional GPIO/servo classes	State-to-pin mapping and physical acknowledgement
Timing	VirtualClock and MonotonicClock interfaces	Separate control scheduling and deadline measurements
Persistence	SQLite event and state transaction	Power-loss and incomplete-actuation reconciliation

Required bench tools are a digital multimeter, logic analyzer and USB power meter; an oscilloscope is useful for supply transients. Use BCM numbering, verify 3.3 V GPIO levels and level-shift 5 V echoes. A servo needs a suitable separate supply and common signal ground. The lighting demonstration uses a low-voltage LED, and the gate is an unloaded bench mechanism.

The shipped app does not activate GPIO. Optional adapter classes are integration building blocks. A target runner must timestamp real acquisitions, disable synthetic acknowledgements, map outputs, and keep interlock checks independent of inference. Direct HX711 support is not included; weight expects a calibrated serial bridge.

Validate in stages: inspect disconnected rails; calibrate sensors; inject wiring and sensor faults; measure LED/servo command and stop timing; run sustained capture under load; interrupt power during writes and movement; then collect deployment data split by complete recording and location. Every physical test remains pending.

Hardware sources: Raspberry Pi computer documentation and gpiozero input/output API documentation. Full URLs, pin notes and the integration checklist are in docs/hardware.md.

Dashboard demonstration and learning

Panel	Interpretation
Input and detections	Uploaded image/recorded frame with boxes, or a clearly labeled synthetic scene.
Observation	Controller state, distinct from detector confidence.
Count and actuator	Candidate count/free slots/acknowledged disposals; virtual output only.
Last inference	Measured model/preprocessing time on this host; not Pi latency.
Latest decision and sensors	Reason, validity and current virtual sensor values.
Observation history	Count versus explicit simulation time; duplicate timestamps may overlap.
Experiment controls	Fault scenarios, manual requests, threshold and applicable interval controls.
Event ledger	Latest 30 rendered events from a 500-event export window; database retains all events.
Recorded benchmarks	Frozen historical results and JSON export; not recomputed by dashboard controls.

Click stationary and observe OCCUPIED / LIGHTS ON after ten simulated minutes. Click poster and see UNCERTAIN / LIGHTS ON. Click camera disconnect to see the failure reason. For vacancy, set fixture count to zero, Check now, then advance ten minutes; grace and two confirmations permit lights off. Manual OFF expires after five simulated minutes.

Start with Python 3.12, create .venv, install requirements.txt and run app.py. Control fixtures need no model files. For AI, install requirements-training.txt and run tools/setup_models.py and tools/fetch_samples.py. The trained waste head is included; source packages omit pretrained backbones and COCO photos. Detailed commands and troubleshooting are in docs/setup-and-demo.md.

Use docs/mentor-guide.md for the code walkthrough, engineering alternatives, ten explained interview questions, modifications and three measured resume bullets. Begin with core.py and its tests, then trace a dashboard action through runtime.py, inference.py and the persistence interface.

Attribution: Ultralytics YOLO11, COCO, TrashNet by Gary Thung and Mindy Yang, and torchvision MobileNetV3. License texts and source links are retained in NOTICE.md, licenses and docs/model-and-data.md. No identity recognition is implemented. The project demonstrates reproducible simulation work; physical deployment and senior professional experience are not claimed.